

Unit 01: Introduction to Linux and CLI Commands

1.1 Introduction to Open-Source Software

- **Definition:** Open-source software (OSS) refers to computer programs whose source code is made publicly available, allowing anyone to view, modify, and distribute it. Unlike proprietary software, where the code is restricted and controlled by a company or individual, OSS encourages collaboration and transparency. Users can customize the software to fit their specific needs, fix bugs, add new features, and share these improvements with the wider community. This approach not only promotes innovation but also ensures that the software evolves continuously, supported by a global network of developers and users.
- **Principles:**
 - Freedom to use, study, modify, and share.
 - Community-driven development and collaboration.
- **Advantages:**
 - Cost-effective (usually free)
 - Flexibility and customization
 - Security through transparency
 - Large community support
- **Examples:** Linux, Apache, MySQL, Firefox

1.2 Unix System Architecture & Linux Differences

- **Unix System Architecture:**
 - Kernel: Core system managing hardware and resources.
 - Shell: Interface between user and kernel.
 - File System: Organizes files and directories.
- **Linux Differences from Unix:**
 - Open-source alternative
 - Runs on various hardware platforms

- Active community development
- **Using a Linux System:**
 - GUI (Graphical) vs CLI (Command-Line Interface)
 - CLI preferred for server/admin tasks due to efficiency.
- **Command Syntax:**

command [options] [arguments]

- Example: `ls -l /home/user` → lists files with details in /home/user.

	Unix	Linux
Origin	Proprietary operating system	Open-source operating system
License	Commercial	Free and open-source
Kernel	Monolithic	Monolithic or hybrid
Software	Specialised and mature ecosystem	Vast collection of open-source software
Customisation	Limited	Highly customisable and adaptable
Cost	Expensive licensing costs	Generally free, with optional commercial support
Stability	Highly stable and reliable	Stable with active development and optimisation
Hardware	Supports various architectures	Supports various architectures
Community	Established support and resources	Active community-driven development

1.3 File Management

File management in Linux refers to the process of organizing, accessing, creating, modifying, moving, and deleting files and directories using commands in the Command-Line Interface (CLI). Efficient file management is crucial for system administration, data organization, and task automation.

- **Basic Commands:**
 - `ls` → List files and directories
 - `cd` → Change directory
 - `pwd` → Print working directory
 - `mkdir` → Create a directory

- `rmdir` → Remove empty directory
- `cp` → Copy files/directories
- `mv` → Move or rename files/directories
- `rm` → Remove files/directories
- **File Permissions:**
 - `r` → read, `w` → write, `x` → execute
 - `chmod`, `chown`, `chgrp` to manage permissions

1.4 Shell Commands

What is a Shell?

A shell is a program that acts as an interface between the user and the operating system kernel. It interprets the commands entered by the user, communicates them to the kernel for execution, and then displays the output. The shell allows users to interact with the system, run programs, automate tasks, and manage files without needing a graphical interface.

- **Common Shells:** Bash (Bourne Again Shell), Zsh, Ksh.
- **Useful Commands:**
 - `echo` → display message
 - `man` → show manual
 - `which` → locate executable
 - `history` → show previously run commands
- **Command Features:**
 - Pipes `|` → connect output of one command to input of another
 - Redirection `>` `>>` `<` → manage input/output files

1.5 Text Processing

Text processing in Linux refers to the use of commands and utilities to view, manipulate, filter, and analyze text files. It is essential for system administration, log analysis, data extraction, and automation. Linux provides powerful commands that can handle large text files efficiently.

- **Commands:**
 - cat → display file content
 - less / more → view file page by page
 - head / tail → view start/end lines
 - grep → search text patterns
 - wc → count words, lines, characters
 - sort → sort file content
 - uniq → remove duplicate lines
- **Practical Use:** Extract logs, filter outputs, automate reports.

1.6 Job Control

Job control in Linux allows users to manage multiple processes (jobs) running in the shell, whether they are in the foreground or background. It helps in multitasking, pausing/resuming processes, and efficiently managing system resources.

- **Foreground & Background Jobs:**
 - & → run job in background
 - jobs → list jobs
 - fg → bring job to foreground
 - bg → resume job in background
- **Process Management:**
 - ps → list processes
 - kill → terminate process
 - top → monitor running processes dynamically

1.7 Filesystem Concepts

The filesystem in Linux/Unix is a structured way of storing and organizing data on storage devices such as hard drives, SSDs, and USB drives. It provides a hierarchical directory structure that allows easy access, management, and security of files.

- **Hierarchy:**
 - / → root directory

- /home → user directories
- /etc → configuration files
- /var → variable data (logs)
- /usr → applications and utilities
- **Types of Files:**
 - Regular files, directories, symbolic links, devices, sockets
- **Mounting & Partitions:** Using mount and df to manage disks and partitions.

1.8 Package Management

Package management in Linux is the process of installing, updating, upgrading, and removing software packages on a system. Packages are collections of files, binaries, libraries, and metadata that allow applications to run correctly. Package management ensures that software dependencies are resolved, keeping the system organized and stable.

- **Purpose:** Install, update, and remove software.
- **Linux Package Managers:**
 - Debian/Ubuntu: apt, dpkg

```
sudo apt update
```

```
sudo apt install package_name
```
 - RedHat/CentOS: yum, dnf, rpm

```
sudo yum install package_name
```
- **Advantages:** Keeps system software organized, ensures dependencies are met, simplifies updates.